

Voice AI Engineer

Bangalore (In Office). Deploying for customers in the US and UK.

[2care.ai](#) is Hiring: Job Description is here

<https://hire.fomogo.in/careers/d9c1b26d-556f-48c9-a902-3b1d82d4e47d/job/01b88bad-39e5-4b29-9a71-14493c11dc6f>

Step 1. Complete your Engineering Assignment

Role: Voice AI Engineer/ ML & Backend Engineer, 2– 6 YOE

Duration: submission deadline 3 days from receipt .

Stack: Retell or Bolna or Vapi or Livekit

Languages: English and Hindi, fixed, with mid-call code-switching.

Use [Cliniko](#) PMS for clinic creation (Doctors availability and patients info)

Context

You're building a voice AI receptionist for a real clinic with at least two locations. Patients call in, speak naturally, in English or Hindi or both in the same call, and walk away with an appointment booked, rescheduled, or cancelled. No human involved.

The goal is not "it works on a happy-path demo." It's "it works the way a real front desk gets tested," which means messy phrasing, dropped calls, returning patients, and cross-branch scheduling.

Stack requirement

Pick Retell or Bolna. Justify the choice in your README: latency, tool-calling reliability, telephony behavior, multilingual handling, cost, whatever actually drove your decision for this clinic. One platform, fully built and live.

What to build

1. The agent

- Full appointment lifecycle: booking, rescheduling, cancellation, conflict resolution
- A real clinic, sourced not invented: real doctors, departments, branches, slot structures
- At least two branches, since several of the required scenarios below depend on cross-branch logic
- English and Hindi, including code-switching mid-sentence, handled through actual multilingual ASR/LLM/TTS capability, not a translation lookup table
- **Book as fast as possible.** Never ask for information the caller already gave or that the agent can reasonably infer. Every turn should move the call toward a completed booking.

2. Required scenarios

Your agent must correctly resolve natural, underspecified time references against live availability, not guesswork:

- "Do you have anything on Dec 13 around 1?"
- "Mondays and Wednesdays work well for me"
- "In the afternoon after I get off work, around 4:30"
- "Any Thursday morning is great"

And it must explicitly handle the following, each a real failure mode, not a hypothetical:

- **Returning patient, no context:** a returning patient calls in and the agent should recognize them and pull prior context, not treat them as new.
- **Missed outbound call, callback:** an outbound call goes unanswered; when the patient calls back, the agent should recognize it as a callback and carry the original context forward, not start cold.
- **Stale availability from memory:** when the caller asks for a different time, the agent must re-run the availability tool call against live data, not answer from an earlier tool result still sitting in context.
- **Earliest-slot search across branches/practitioners:** asked for the earliest same-day slot, the agent must search correctly across all practitioners and branches. It should not anchor on one practitioner's later slot, or miss a closer same-day opening at another branch, or claim there's nothing available when there is.
- **Branch-specific triage reliability:** querying specialty availability at a named branch must work consistently, not fail intermittently.
- **Dropped call recovery:** if a call disconnects, and the patient calls back, the agent should acknowledge the drop and resume from where the call left off, not restart the conversation.

3. The backend

- A real datastore, not an in-memory dict or flat file, with conflict/double-booking checks enforced at write time

- A mock EHR/PMS write-back API (yours to build) that tool calls hit after a confirmed booking, with idempotency and a defined behavior when that call fails. You can use [Cliniko](#) PMS OR Any other PMS. Cliniko offers 30 days free trial.
- A clean, well-scoped tool-calling schema between your agent and your backend

4. The eval harness

- Multi-turn scripted conversations covering the scenarios above, not single-turn checks
- Metrics reported per language (English / Hindi), not blended
- Latency broken down by component where possible (ASR, LLM, TTS, network)
- A turns-to-completion metric: how many turns to a confirmed booking, and how often a redundant question gets asked
- Re-runnable from a clean clone. We will re-run it independently.

Document why you picked these dimensions, what the numbers actually tell you, and where the harness gives false confidence.

Deliverables

- **GitHub repo:** clean, runnable with reasonable setup effort, backend plus your platform integration
- **README:** what you built, key design choices including your stack justification, latency numbers, multilingual approach, known limitations

Submission

Send one email with:

- A link to your write-up. Your README is fine for this, or a Notion / Google Doc if you prefer, just send a link, not pasted text in the email body
- Your GitHub repo link
- Your prompt and prompt logic
- The phone number we should call to test it live

Send all of it to tech@2care.ai, p@2care.ai, and s@2care.ai.

Weighting

Area

Weight

Works end to end on real data, in both languages	Core
Handles the required scenarios above correctly	Core
Never asks a redundant question; books quickly	Core
Latency reasoning reflected in the actual build	Core
Eval harness measures something real, per language	Core
Stack choice is justified with real reasoning, not just "it was easier"	Core
Prompt is clean and holds up in Hindi, not just English	Core
We can run it ourselves	Core

Constraints

- Make sure the voice is very realistic. Interruption handling is good.
- No hardcoded translation dictionaries. If we say something in Hindi that isn't in a canned list, it needs to actually work.
- Your deployment must be live and independently callable, hitting a real backend and database.
- If we can't call the agent ourselves and have it work, it doesn't count.

Test Cases

These are real failure modes we've observed testing voice AI receptionists in production, across multiple live clinics. They're not hypothetical edge cases — they're the specific things that break when a real front desk gets tested. We will check for these on the live number you provide, in addition to the required scenarios above. None of these require exotic engineering — they're mostly about state retention, live-data checks instead of cached assumptions, and testing beyond the happy path. We'd rather see four of these handled well than a demo that only works on the scripted examples above.

Identity & call handling

- The agent should always capture the caller's full name before completing a booking, even if the phone number is already recognized in the system. A booking should never go through anonymously.
- If two different patients share one phone number (e.g. a family line), the agent should ask for the name first to disambiguate, rather than assuming the wrong patient or asking for unnecessary extra verification.
- If a call disconnects mid-conversation and the patient calls back, the agent should recognize this as a continuation, acknowledge the drop briefly, and resume — not restart the intake from scratch.

Scheduling correctness

- When a caller asks for the earliest available slot, the agent should genuinely compare across all practitioners and both branches before answering — not default to one practitioner's schedule and miss a closer opening elsewhere.
- If a slot is offered and then found to be taken by the time of booking, the agent should handle this gracefully with a live re-check, not confirm a booking that then silently fails.
- The location or branch confirmed back to the caller out loud must always match the branch actually being booked in the backend.
- Same-day bookings should respect any required buffer time between appointments, not pack slots back-to-back if the clinic's data specifies a gap.
- Rescheduling should book the correct new slot every time, and any cancellation/rescheduling fee should only be mentioned when the change actually falls within the policy window that triggers a fee — not stated by default on every reschedule.

Date, time, and locale accuracy

- A same-day appointment must always be communicated with the correct local date. Watch specifically for UTC/timezone conversion bugs that shift “today” to “tomorrow.”
- If your clinic setup involves any pricing or currency mentions, make sure the correct currency is used consistently for the clinic's actual location.

Language handling

- In a single-language turn (pure English or pure Hindi, no code-switch cue from the caller), the agent should not spontaneously drift into the other language or insert stray words from it.
- Code-switching should work in both directions within the same sentence, and the agent's own code-switches should sound natural, not like two languages stitched together.

Conversational quality

- The agent should never re-ask a question the caller already answered earlier in the same call — this signals broken state tracking, not just a minor annoyance.
- While a tool call (availability lookup, booking write) is in flight, the agent should not stutter, repeat filler words, or cut off mid-word. Latency should be masked with a natural holding phrase.

- If a patient or doctor name is stored in all-caps in your data, the agent should still pronounce it naturally — not spell it out letter by letter.
- Interruption handling should feel natural: a caller cutting in mid-sentence should not confuse the agent's state or cause it to lose its place.

Escalation & human handoff

- If a caller asks her they're speaking to a bot or a human, or insists on speaking with a person, the agent should handle this gracefully rather than deflecting indefinitely or pretending otherwise.
- If a caller asks for a human, raises a clinical concern, or has another issue outside the booking flow, the agent should log it as needing follow-up rather than just dropping it. Do not imply an immediate live transfer is happening unless one actually is — set the expectation that someone will call back.

All the best

Only after you submit the assignment, Book the technical interview with Bala :

<https://cal.com/balaguhanesh/30min>

In the interview you will be discussing your work on how / what you built in your assignment and your about previous experiences. So **DO NOT book the interview until you have completed/submitted the assignment.**